FHIR Retriever: User and Technical Documentation

1. Purpose

This project downloads a selected FHIR R4 Patient cohort from
‘http://hapi.fhir.org/baseR4‘, stores Patient, Condition, Observation, and Encounter
data in SQLite and CSV files, and exposes the stored data through an HTTP API.

The normal flow is:

‘‘‘text
FHIR endpoint -> retrieval -> pseudonymization -> SQLite/CSV -> analysis/API
‘‘‘

2. Files

File	Purpose
‘fhir_retriever.py‘	FHIR client, cache/checkpoint handling, database loading, CSV export, command-line interface.
‘fhir_analyse.py‘	Reads numeric Observations from SQLite and calculates descriptive statistics.
‘fhir_api.py‘	FastAPI HTTP service over the stored SQLite data.
‘etl.py‘	Compose ETL entry point: retrieve the configured cohort, load it, then create an analysis result.
‘test_fhir_retriever.py‘	Retrieval, cache, database, and resource-selection tests.
‘test_fhir_analyse.py‘	Statistics and analysis-output tests.
‘test_fhir_api.py‘	HTTP API smoke tests.
‘test_encounter_link_repair.py‘	Migration test for legacy Encounter/Observation ID linking.
‘Dockerfile‘, ‘docker-compose.yml‘, ‘.env.example‘	Container build, orchestration, and environment configuration.
‘generate_documentation_pdf.py‘	Regenerates this PDF from ‘docs/TECHNICAL_USER_DOCUMENTATION.md‘.

3. Setup

Install dependencies (a virtual environment is recommended):

‘‘‘bash
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
‘‘‘

Set the pseudonymization key in the current shell:

‘‘‘bash
export FHIR_PSEUDONYMIZATION_KEY=’replace-with-a-long-private-random-value’
‘‘‘

The key is not a Patient ID. It is the secret used by HMAC-SHA256 to turn source FHIR identifiers into stable pseudonyms. The same secret must be used whenever the same dataset is updated, otherwise new pseudonyms will not match older rows, and single-Patient commands will not recognize previously cached data for that Patient.

4. Main Commands

Retrieve all Patients found by the Patient search:

‘‘‘bash
python -m fhir_retriever --all-patients --output-dir fhir_output

```

Retrieve only the first N Patients:

```bash
python -m fhir_retriever --all-patients --limit 100 --output-dir fhir_output
```

Retrieve only one Patient by FHIR ID:

```bash
python -m fhir_retriever --patient-id sindhu-syn-000004 --output-dir fhir_output
```

Retrieve one Patient plus selected related resources. `--observation`, `--encounter`, and `--condition` can be combined in any subset:

```bash
python -m fhir_retriever \
  --patient-observations-encounters sindhu-syn-000004 \
  --output-dir fhir_output \
  --condition --observation --encounter
```

The same modifiers work with `--all-patients` to fetch related resources for every Patient:

```bash
python -m fhir_retriever --all-patients --output-dir fhir_output --observation
```

Use `--refresh` only when deliberately fetching again from HAPI. Without it, completed work is served from the local cache/checkpoint where possible.

Every command's JSON output includes a `patient_pseudonyms` map for any Patient ID given explicitly, so the pseudonym stored for that Patient is always visible:

```json
{
  "resources": {"Patient": 1, "Condition": 0, "Observation": 0, "Encounter": 0},
  "failures": [],
  "patient_pseudonyms": {"sindhu-syn-000004": "PAT-1cd821a6f94aa7e10f01"}
}
```

5. Retrieval Code Walkthrough

`Pseudonymizer`

`Pseudonymizer` receives `FHIR_PSEUDONYMIZATION_KEY` from the environment. `identifier(resource_type, raw_id)` computes an HMAC-SHA256 digest over the resource type and source ID, then prefixes a shortened digest with `PAT`, `OBS`, `ENC`, or another resource prefix. The HMAC prevents someone with only the output database from reversing the pseudonym.

`patient_offset(patient_id)` derives a stable integer from the same key, in the range -365 to +365 days. `shift_date(value, offset_days)` parses an ISO date or timestamp and applies that offset. If shifting would move a date outside the valid calendar range (an edge case seen with a small number of public HAPI test records near year 1 or year 9999), the original value is kept unshifted instead of raising an error, so one unusual record cannot crash an entire retrieval.
```

‘FHIRRetriever.__init__‘

The constructor validates retry, page-size, and limit settings, opens the local cache/checkpoint paths, configures the HTTP session, opens SQLite, and hydrates the database from the full local cache. No network request happens during construction.

‘_make_session‘

This method creates a ‘requests.Session‘ with an ‘HTTPAdapter‘ and ‘urllib3.Retry‘ policy. It retries only GET requests for transient status codes such as 429 and 503, uses exponential backoff, honors ‘Retry-After‘, and supplies a descriptive User-Agent.

‘_fetch_pages‘

FHIR search results are Bundles. This loop requests one Bundle page, validates the response, persists its resources, reads the Bundle ‘next‘ link, and repeats until there is no next page or an optional resource limit is reached. A request error, invalid JSON, or invalid Bundle becomes a ‘RetrievalFailure‘; the partial resources already persisted remain available.

‘_pseudonymize_resource‘

This method runs before data is persisted. It copies a FHIR resource, replaces resource IDs and references with stable pseudonyms, shifts relevant dates, and removes Patient identifiers, telecom details, addresses, contacts, and communication preferences. Patient family and given names are extracted from the FHIR ‘name‘ field into internal fields before the original ‘name‘ array is removed, so they can still be stored in the ‘patients‘ table. Observation and Encounter references are rewritten so relationships survive.

‘_sync_single_patient_scope‘

Called whenever a command targets one specific Patient (‘--patient-id‘, ‘--patient-observations-encounters‘, and the underlying ‘get_patient‘/‘get_*_for_patient‘ functions). It removes rows for any other Patient from the ‘patients‘, ‘conditions‘, ‘observations‘, and ‘encounters‘ tables, then re-syncs only that Patient’s cached data. This keeps single-Patient commands scoped to the requested Patient, even when the same ‘--output-dir‘ was previously used for ‘--all-patients‘ or a different Patient.

‘_load_related‘

This method loops over the requested Patients and resource types. It performs a separate FHIR search for each combination, for example ‘Observation?patient=<source-id>‘. The checkpoint uses pseudonymized query identifiers, while source IDs only remain in process memory long enough to perform requests.

‘FHIRDatabase.sync‘

‘sync‘ receives de-identified resources and performs SQLite inserts/upserts. Stable primary keys for Patients, Conditions, Observations, and Encounters mean a later run updates rows instead of adding duplicates, so loads are idempotent.

‘export_csv‘

After a retrieval operation, SQLite tables are exported to CSV, overwriting any existing files. Headers are read from ‘PRAGMA table_info‘, so CSV column order follows the active SQLite schema. Export happens after every database sync during a run, not only once at the end.

6. Database Schema

‘‘‘sql

```
patients(patient_id, family_name, given_name, gender, birth_date, date_shift_days)
conditions(condition_id, clinicalStatus, verificationStatus, category, condition,
           condition_code, patient_id, encounter_id, onsetDateTime, recorded_Date)
observations(observation_id, patient_id, encounter_id, observation_type,
             observation_code, observation_subtype, effective_date_time, issued,
             value, unit, value_code)
encounters(encounter_type, encounter_id, start, end, patient_id)
```

`patients.family_name` and `patients.given_name` are populated from the FHIR `Patient.name` field. `patient_id` itself is always a pseudonym, and `birth_date` is shifted by `date_shift_days`, so names are stored alongside a de-identified identifier and date rather than the original ones.

The intended relationships are:

```text
patients.patient_id -> conditions.patient_id
patients.patient_id -> observations.patient_id
patients.patient_id -> encounters.patient_id
encounters.encounter_id -> observations.encounter_id
encounters.encounter_id -> conditions.encounter_id
```

SQLite stores ISO dates as `TEXT`; this is standard SQLite practice because it does not provide a dedicated date storage class.

7. Manual Database Checks

Query the database directly with Python if the `sqlite3` command-line client is not installed:

```bash
python3 -c "
import sqlite3
conn = sqlite3.connect('fhir_output/fhir_resources.sqlite3')
conn.row_factory = sqlite3.Row
for row in conn.execute('SELECT * FROM observations LIMIT 10'):
    print(dict(row))
"
```

If the `sqlite3` CLI is available:

```bash
sqlite3 fhir_output/fhir_resources.sqlite3
```

List tables:

```sql
.tables
```

Inspect a schema:

```sql
.schema observations
```

Count resources:

```sql
SELECT 'patients', COUNT(*) FROM patients
UNION ALL SELECT 'conditions', COUNT(*) FROM conditions
UNION ALL SELECT 'observations', COUNT(*) FROM observations
UNION ALL SELECT 'encounters', COUNT(*) FROM encounters;
```

Check Observation-to-Encounter joins:

```sql
SELECT o.observation_subtype, o.value, o.unit, e.encounter_type
FROM observations AS o
LEFT JOIN encounters AS e ON e.encounter_id = o.encounter_id
LIMIT 20;
```

Check for duplicate primary IDs, confirming idempotent loads:

```sql
SELECT observation_id, COUNT(*)
FROM observations
GROUP BY observation_id
HAVING COUNT(*) > 1;
```

No rows from the final query means Observation loading is idempotent.

8. Analysis

Example:

```bash
python -m fhir_analyse \
  --obs-value "Alanine Aminotransferase" \
  --group-by sex \
  --output-dir fhir_output
```

`--obs-value` matches either the Observation display (`observation_subtype`) or code (`observation_code`). `--group-by` accepts `age-band`, `sex`, or `encounter-type`; the `encounter-type` grouping joins `observations.encounter_id` to `encounters.encounter_id`. The output contains count, mean, median, standard deviation, minimum, and maximum. It is printed and written to `<output-dir>/analysis.txt`.

9. HTTP API

Start locally after installing FastAPI:

```bash
FHIR_DATABASE_PATH=fhir_output/fhir_resources.sqlite3 \
uvicorn fhir_api:app --host 0.0.0.0 --port 8000
```

Endpoints:

```text
GET /health
GET /patients?limit=100&offset=0
GET /patients/{patient_id}
GET /observations?patient_id=PAT-...&observation_code=1742-6&encounter_id=ENC-...
GET /analysis/observations?observation=1742-6&group_by=sex
GET /docs
```

‘‘‘

‘GET /patients‘ fetches live from the FHIR endpoint whenever the local database has fewer Patients than ‘offset + limit‘ (equivalent to running ‘fhir_retriever --all-patients --limit N‘), then serves the requested page from SQLite. Pass ‘refresh=true‘ to force a fresh fetch even when enough Patients are already cached.

Patient responses (‘/patients‘ and ‘/patients/{patient_id}‘) never include ‘family_name‘ or ‘given_name‘, even though those columns exist in the local database; they are stripped from the API response only.

FastAPI returns JSON and OpenAPI documentation. Invalid query bounds return 422, missing Patients return 404, and a missing/unavailable database returns 503.

10. Testing

Run focused tests:

‘‘‘bash
python -m unittest -v test_fhir_retriever.py
python -m unittest -v test_fhir_analyse.py
python -m unittest -v test_fhir_api.py
python -m unittest -v test_encounter_link_repair.py
‘‘‘

Run the full suite once the dependencies in ‘requirements.txt‘ are installed:

‘‘‘bash
python -m unittest -v
‘‘‘

The retrieval tests use fake FHIR responses; they do not call HAPI. Tests validate pagination, retries, partial failures, cache-only reruns, single-Patient scoping, idempotent upserts, API status codes, CSV exports, and relationships.

11. Deployment (Docker or Podman)

‘‘‘bash
cp .env.example .env
Edit .env and set FHIR_PSEUDONYMIZATION_KEY
podman compose up --build
or, with Docker installed:
docker compose up --build
‘‘‘

Compose uses a named ‘fhir-data‘ volume, shared read-write by ‘etl‘ and ‘api‘. ‘etl‘ retrieves the configured cohort, writes SQLite/CSV data, and exits after a successful run. ‘api‘ starts only after ‘etl‘ completes successfully and exposes port 8000.

‘.env‘ controls the cohort:

- ‘FHIR_COHORT_MODE=single‘ (default): loads only ‘FHIR_COHORT_PATIENT_ID‘.
- ‘FHIR_COHORT_MODE=all‘: loads up to ‘FHIR_PATIENT_LIMIT‘ Patients (default ‘10‘), useful for testing ‘/patients?limit=N‘ with more than one Patient.

12. Privacy Limits

Pseudonymization reduces exposure but is not anonymization. Patient ‘family_name‘/‘given_name‘ are retained in the local database and CSV exports (though excluded from API responses), and ‘patient_id‘, dates, and clinical references are pseudonymized/shifted rather than removed entirely. This does not protect against a party holding the HMAC secret, external linkage attacks, or rare combinations of dates,

measurements, or diagnoses. Protect the secret and the local database/CSV files, restrict access, and apply a privacy review before publishing or sharing data.